



ADS1242 — Mensageria e Streams em Aplicações

Comunicação em Tempo Real

HTTP Polling vs **WebSocket**

Entendendo a diferença e quando usar cada abordagem

Agenda

01 O problema da comunicação em tempo real

02 HTTP Polling — como funciona e seus limites

03 Demonstração prática — projeto Polling

04 WebSocket — protocolo e handshake

05 STOMP + Spring Boot — implementação

06 Demonstração prática — projeto WebSocket

07 Comparativo e quando usar cada abordagem

O Problema

Como o cliente sabe que há dados novos no servidor?



Leilão ao vivo

Novo lance de outro usuário



Cotações financeiras

Preço atualizado a cada segundo



Jogo multiplayer

Posição dos outros jogadores



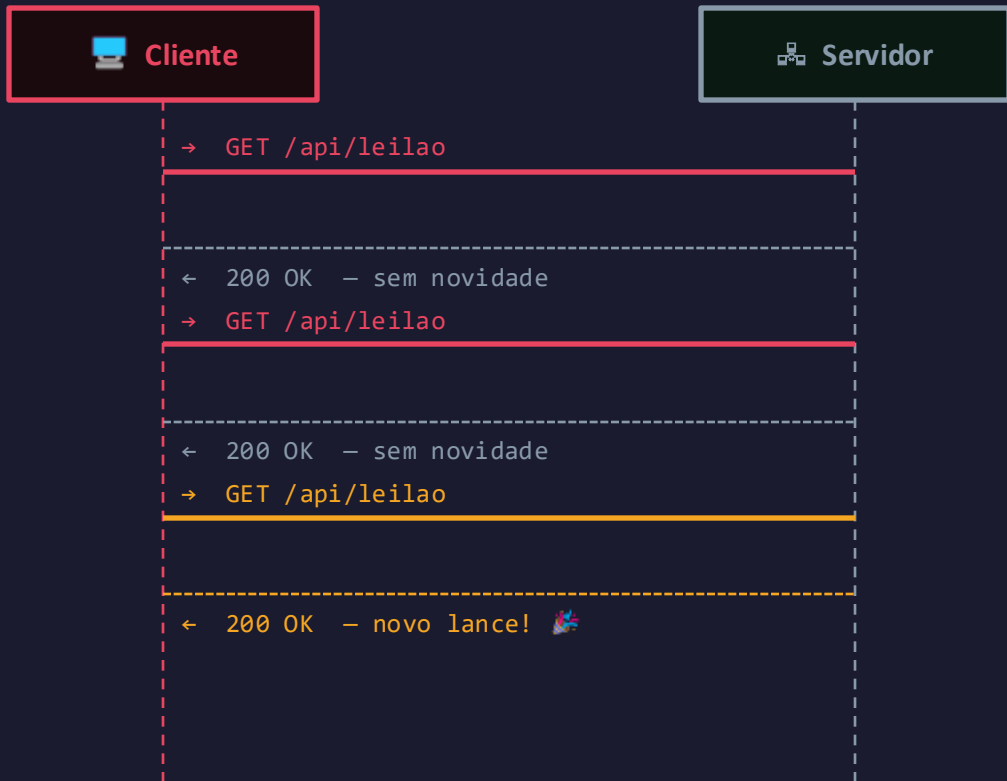
Rastreo de entrega

Posição do entregador no mapa

HTTP foi projetado para requisição→resposta. **Como receber dados sem perguntar o tempo todo?**

HTTP Polling

Como funciona



Neste projeto: `GET /api/leilao` polling a cada 2 segundos



A cada 2s, com ou sem novidades



Mesmos headers HTTP em cada requisição



Atraso de até 2s para ver o novo lance

Problemas do Polling



Latência inevitável

Se o intervalo de polling for de 2s, o usuário pode esperar até 2 segundos para ver uma atualização, mesmo que ela já tenha acontecido no servidor.

até 2s

de atraso



Tráfego desnecessário

Cada requisição carrega cabeçalhos HTTP completos (cookies, User-Agent, Accept...). Em 5 minutos com 30 usuários = 4.500 requisições — mesmo sem novidades.

~72B

headers por req.



Escalabilidade baixa

Com 1000 usuários e polling de 2s, o servidor recebe 500 requisições/segundo apenas para verificar se há atualizações — mesmo em um leilão inativo.

500/s

reqs com 1k usuários

● DEMONSTRAÇÃO — HTTP Polling

http://localhost:8080

- 1 Observe o contador “Requisições HTTP” crescendo a cada 2 segundos
- 2 Fique 1 minuto SEM dar lance — veja o contador “Reqs. Inúteis” disparar
- 3 Abra DevTools → Network e filtre por `leilao` — veja as requisições se repetirem
- 4 Dê um lance — note que o outro participante leva até 2s para ver a atualização
- 5 Ao fim do leilão: observe quantas requisições foram feitas vs quantos lances reais houve

💡 *Dica: abra em duas abas ou dois navegadores para simular dois licitantes*

WebSocket

Canal bidirecional e persistente sobre TCP

WebSocket é um protocolo (RFC 6455) que **inicia com um handshake HTTP** e depois **faz upgrade para uma conexão TCP full-duplex persistente**. O servidor pode enviar dados a qualquer momento, sem aguardar uma requisição do cliente.

✗ HTTP

1 req → 1 resp → conexão fechada

✓ WebSocket

1 handshake → conexão aberta indefinidamente

🔑 Handshake: HTTP 101 Switching Protocols → upgrade: websocket

WebSocket — Fluxo de Mensagens



STOMP sobre WebSocket

Simple Text Oriented Messaging Protocol

Aplicação (Lógica de Negócio)

@MessageMapping @SendTo SimpMessagingTemplate

STOMP (Protocolo de Mensagens)

SUBSCRIBE /topic/leilao SEND /app/lance

WebSocket (Transporte)

Canal full-duplex • Frames binários ou texto

TCP/IP (Rede)

Conexão persistente • Porta 80 / 443

SUBSCRIBE

Cliente se inscreve em um tópico

/topic/X

Tópico público — recebido por TODOS

SEND

Cliente envia mensagem ao servidor

/app/X

Roteado para @MessageMapping

SimpMessagingTemplate

Servidor faz broadcast programático

Spring Boot — Implementação WebSocket

WebSocketConfig.java

```
@EnableWebSocketMessageBroker

public class WebSocketConfig ... {

    registry.enableSimpleBroker("/topic");

    registry.setApplicationDestination
        Prefixes("/app");

    registry.addEndpoint("/ws-leilao")

        .withSockJS(); }
```

LeilaoController.java

```
@RequestMapping("/lance")
@SendTo("/topic/leilao")

public LeilaoState receberLance(Lance l) {

    // validar e salvar o lance...

    return estado; // broadcast automático

}
```



Endpoint de conexão

/ws-leilao (SockJS + WS)



Prefixo cliente→servidor

/app/* → @MessageMapping



Prefixo broadcast

/topic/* → todos os subscribers



@SendTo

Retorno do método vira broadcast automático



DEMONSTRAÇÃO — WebSocket

`http://localhost:8081`

- 1 Note o badge `Conectado via WS` — conexão já estabelecida ao abrir a página
- 2 Abra DevTools → Network → clique na aba `WS` — veja apenas 1 handshake
- 3 Clique na conexão → aba `Messages` — observe os frames chegando a cada 1s
- 4 Dê um lance — veja o valor atualizar INSTANTANEAMENTE nos outros navegadores
- 5 Compare `Msgs Recebidas` vs `Lances` — o tráfego é proporcional ao que muda



Abra os dois projetos lado a lado e compare visualmente o Network tab

Comparativo

Critério	HTTP Polling	WebSocket
Conexões por cliente	1 a cada N segundos	1 permanente
Tráfego sem novidades	Contínuo e constante	Zero
Latência de atualização	Até N segundos	< 10ms
Overhead de headers	Sim, em cada request	Mínimo (frames)
Escalabilidade	Baixa	Alta
Complexidade impl.	Simples	Média
Suporte em navegadores	Universal	99%+ (+ SockJS fallback)

Quando usar cada abordagem?

Use Polling quando...

- Dados mudam raramente (uma vez por hora)
- Simplicidade é prioridade no backend
- A aplicação já usa REST e não justifica migração
- Poucos usuários simultâneos
- Ambientes corporativos que bloqueiam WS

Use WebSocket quando...

- Dados mudam frequentemente (segundos ou menos)
- Múltiplos usuários precisam ver o mesmo estado
- Latência baixa é requisito (jogos, leilões, trading)
- Chat, notificações ou colaboração ao vivo
- Dashboards de monitoramento em tempo real

Recapitulando



Polling: cliente pergunta repetidamente — simples, mas desperdiçador



WebSocket: servidor avisa na hora — eficiente, bidirecional, de baixa latência



STOMP organiza as mensagens WS com tópicos e prefixos de roteamento



Spring Boot: `@EnableWebSocketMessageBroker` + `@MessageMapping` + `@SendTo`



A escolha depende da frequência de atualização e do número de usuários